



ĐẠI HỌC THÁI NGUYÊN  
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG**  
THAI NGUYEN UNIVERSITY OF INFORMATION AND COMMUNICATION TECHNOLOGY



[www.ictu.edu.vn](http://www.ictu.edu.vn)



# Bài 1: Tổng quan quan về lập trình HĐT

1

- ❑ 1.1. Các phương pháp tiếp cận trong lập trình
- ❑ 1.2. Lập trình hướng đối tượng, một số ngôn ngữ lập trình HĐT cơ bản



## 1.1. Các phương pháp tiếp cận trong lập trình

2

- ☐ Lập trình tuyến tính
- ☐ Lập trình hướng chức năng
- ☐ Lập trình hướng đối tượng
- ☐ So sánh với các phương pháp



# 1.1. Các phương pháp tiếp cận trong lập trình

3

1. Lập trình tuần tự (Phương pháp/Tiếp cận tuần tự)  
Đơn luồng; Đơn giản;  
Tuần tự.
2. Lập trình có cấu trúc (Phương pháp/Tiếp cận hướng chức năng)  
Tập trung vào chức năng, nhiệm vụ;  
Phân rã làm mịn dần top-down; Chia để trị
3. Lập trình hướng đối tượng (Phương pháp/Tiếp cận hướng đối tượng)  
Tập trung vào các đối tượng;  
Xem hệ thống là tập các thực thể và đối tượng;  
Đặt trọng tâm vào dữ liệu (thực thể)



# Lập trình tuyến tính

4

## ❖ Đặc trưng cơ bản

- Đơn giản: Chương trình được tiến hành đơn giản theo lối tuần tự, không phức tạp.
- Đơn luồng: Chỉ có một luồng công việc duy nhất, các công việc được thực hiện tuần tự trong các luồng đó.

## ❖ Ưu điểm: Chương trình đơn giản, dễ hiểu.

## ❖ Hạn chế:

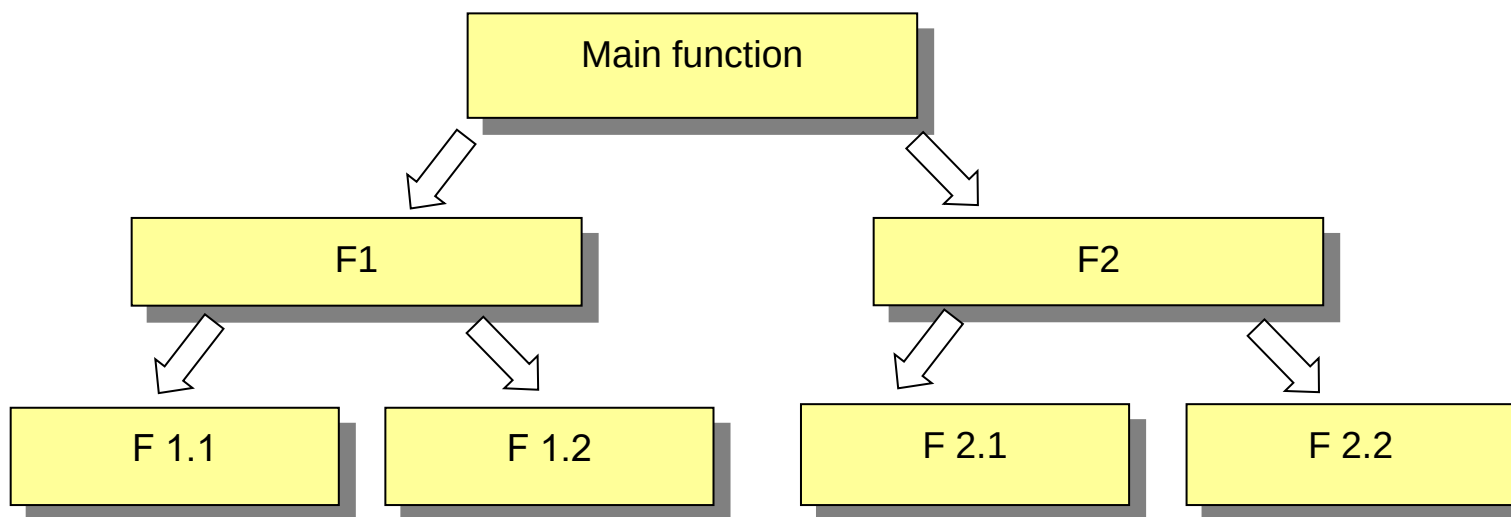
- Không thể dùng lập trình tuyến tính để giải quyết các bài toán phức tạp.
- Ngày nay, lập trình tuyến tính chỉ tồn tại trong phạm vi các module nhỏ nhất của các phương pháp lập trình khác.

## ❖ Ngôn ngữ lập trình tuyến tính không có khả năng kiểm soát phạm vi nhìn thấy của các dữ liệu. Mọi dữ liệu trong chương trình đều là dữ liệu toàn cục nghĩa là chúng có thể bị sửa đổi ở bất kỳ phần nào của chương trình.



## ❖ Đặc trưng cơ bản

- Dựa vào chức năng nhiệm vụ là chính.
- Phân rã các chức năng theo cách tiếp cận Top-Down và làm mịn dần.
- Các đơn thể (hàm; thủ tục; mô-đun; ...) trao đổi với nhau bằng cách truyền tham số hoặc sử dụng dữ liệu chung.





# Lập trình hướng chức năng

6

## ❖ Hạn chế

- Sản phẩm hình thành từ giải pháp này khó bảo trì
  - Mọi chức năng đều chia sẻ khối dữ liệu lớn
  - Các chức năng phải hiểu rõ dữ liệu được lưu trữ thế nào
  - Khi thay đổi cấu trúc dữ liệu kéo theo thay đổi mọi hàm liên quan
- Tiến trình phát triển không ổn định; Khả năng tái sử dụng bị hạn chế và Không hỗ trợ cơ chế kế thừa
  - Thay đổi yêu cầu kéo theo thay đổi các chức năng
  - Rất khó bảo toàn kiến trúc thiết kế ban đầu khi hệ thống tiến hóa
- Các đơn thể (hàm; thủ tục; mô-đun; ...) trao đổi với nhau bằng cách truyền tham số hoặc sử dụng dữ liệu chung.



# Lập trình hướng đối tượng

7

## ❖ Chiến lược phát triển phần mềm hướng đối tượng là quan sát thế giới như tập các đối tượng

- Các đối tượng tương tác và cộng tác với nhau để hình thành hành vi mức cao

## ❖ Các tính chất của đối tượng

- Đối tượng có thể là:
  - thực thể nhìn thấy được trong thế giới thực (trong pha phân tích yêu cầu)
  - biểu diễn thực thể hệ thống (trong pha thiết kế)
- Đối tượng có trách nhiệm quản lý trạng thái của mình, cung cấp dịch vụ cho đối tượng khác khi có yêu cầu
  - do vậy, dữ liệu và hàm cùng gói trong đối tượng
- Chức năng hệ thống:
  - các dịch vụ được yêu cầu và cung cấp như thế nào giữa các đối tượng, không quan tâm đến thay đổi trạng thái bên trong đối tượng
- Các đối tượng được phân thành class
  - Các đối tượng thuộc cùng lớp đều có đặc tính (thuộc tính và thao tác) chung

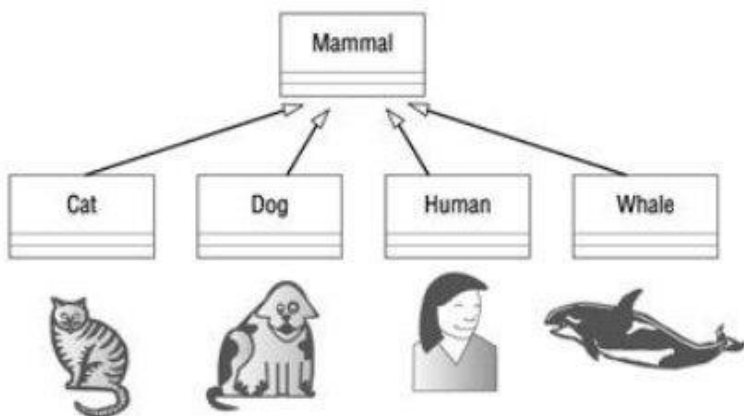


# Lập trình hướng đối tượng

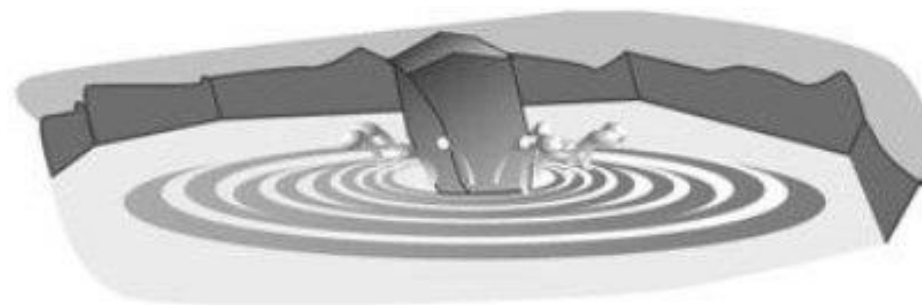
8

## ❖ Ưu điểm

- Tiệm cận hướng đối tượng tập trung vào cả thông tin và hành vi
- Cho khả năng xây dựng hệ thống mềm dẻo, “co giãn”
- Phương pháp này dựa trên các nguyên tắc sau
  - Tính gói
  - Kế thừa
  - Đa trị



Natural Model



Lake Model



# So sánh

9

|                                  | Lập trình tuyến tính                                    | Lập trình hướng chức năng                                    | Lập trình hướng đối tượng                                                    |
|----------------------------------|---------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Khái niệm cơ bản</b>          | Các câu lệnh được thực thi tuần tự, từ trên xuống dưới. | Tập trung vào các hàm và quá trình xử lý dữ liệu.            | Tập trung vào các đối tượng và lớp để tổ chức dữ liệu và hành vi.            |
| <b>Tổ chức mã nguồn</b>          | Các câu lệnh liên tục, ít tổ chức.                      | Các hàm và các đơn vị chức năng được phân chia rõ ràng.      | Các lớp và đối tượng tổ chức dữ liệu và hành vi thành các nhóm có liên quan. |
| <b>Đặc điểm chính</b>            | Quy trình đơn giản, dễ hiểu nhưng ít linh hoạt.         | Tính chất bất biến, tránh trạng thái.                        | Tính đóng gói, kế thừa, và đa hình.                                          |
| <b>Quản lý trạng thái</b>        | Trạng thái toàn cục, khó theo dõi.                      | Trạng thái hạn chế, thường không lưu trữ trạng thái.         | Trạng thái được lưu trữ trong các đối tượng.                                 |
| <b>Tính mở rộng và bảo trì</b>   | Khó mở rộng và bảo trì khi chương trình phức tạp.       | Có thể mở rộng tốt hơn, nhưng vẫn phụ thuộc vào tổ chức hàm. | Dễ dàng mở rộng và bảo trì nhờ vào cấu trúc lớp và đối tượng.                |
| <b>Tính tái sử dụng mã nguồn</b> | Khó khăn trong việc tái sử dụng mã nguồn.               | Tái sử dụng hàm và module dễ dàng hơn.                       | Tái sử dụng qua kế thừa và đa hình.                                          |
| <b>Quản lý lỗi</b>               | Lỗi thường khó phát hiện do cấu trúc không rõ ràng.     | Lỗi dễ phát hiện hơn nhờ vào tính chất bất biến.             | Lỗi dễ theo dõi hơn nhờ vào cấu trúc lớp và đối tượng.                       |



## 1.2. Lập trình hướng đối tượng, một số ngôn ngữ lập trình HĐT cơ bản

10

- ❑ Các khái niệm cơ bản
- ❑ Một số ngôn ngữ lập trình hỗ trợ lập trình HĐT
- ❑ Cài đặt, thiết lập môi trường phát triển ứng dụng Java



# Các khái niệm cơ bản

11

- ☐ Đối tượng
- ☐ Lớp đối tượng
- ☐ Trừu tượng hóa
- ☐ Bao bọc và che dấu thông tin
- ☐ Kế thừa và mở rộng
- ☐ Đa xạ và nạp chồng
- ☐ Liên kết động
- ☐ Truyền thông điệp



❖ **Đối tượng:** Mỗi đối tượng đều có những thuộc tính hay đặc điểm mô tả cũng như những hành vi riêng biệt nhằm phân biệt nó với các đối tượng khác.





## ❖ Thuộc tính và hành vi/phương thức của đối tượng

- **Thuộc tính (Attribute):** Đặc điểm đặc trưng của đối tượng, thể hiện thông qua những giá trị cụ thể.
- **Phương thức (Method):** Cách thức mà qua đó đối tượng thể hiện sự hoạt động hay chức năng của chúng



Name: **Jack**  
Age: 28  
Weight: 65 kgs

**Actions:**

Walk  
Talk  
Sleep

Đối tượng khách hàng



Model: **Ferrari**  
Colour: Red  
Year: 1995

**Actions:**

Start  
Stop  
Accelerate

Đối tượng Car  
xe mà khách hàng mua



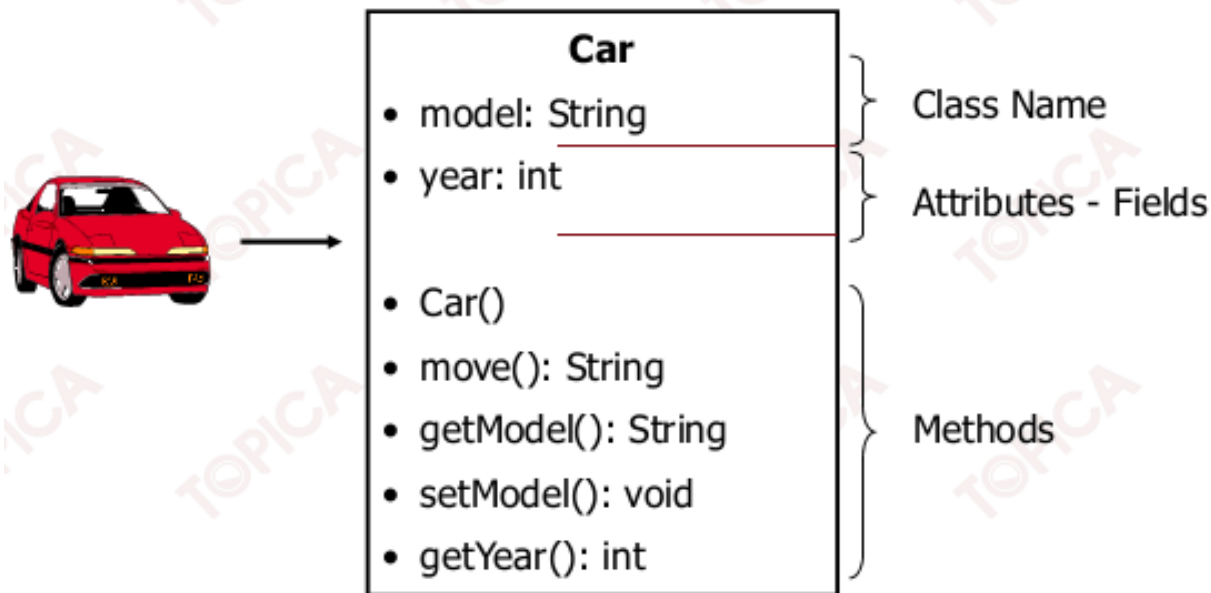
## ❖ Lớp/ Lớp đối tượng (Class)

- **Lớp** là khái niệm dùng để mô tả một nhóm các đối tượng có những thuộc tính, hành vi và những mối quan hệ thông thường tương tự nhau.
- **Lớp đối tượng** còn được gọi với tên ngắn gọn là **Lớp**

=> *Như vậy*: Mỗi đối tượng được coi như là một “thể hiện” của một lớp nào đó với những giá trị thuộc tính cũng như cách thức hoạt động đặc trưng.



❖ **Sơ đồ mô tả lớp (UML):** Sơ đồ lớp (sử dụng UML) mô tả những đặc điểm khái quát của lớp đó, bao gồm: Tên lớp; danh sách các thuộc tính; và danh sách các phương thức.





# Trừu tượng hóa

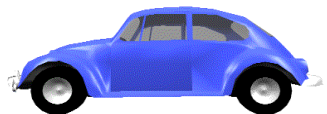
16



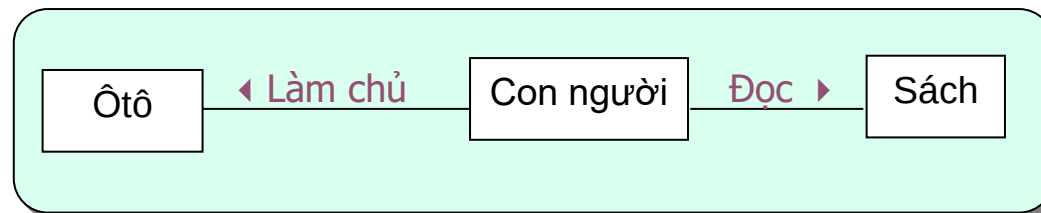
Thế giới thực



Mô hình: Quả địa cầu học sinh



Thế giới thực



Mô hình



- ❖ Cách biểu diễn những đặc tính chính và bỏ qua những chi tiết không quan trọng.
- ❖ Sự mở rộng khái niệm kiểu dữ liệu và cho phép định nghĩa những phép toán trừu tượng trên các dữ liệu trừu tượng.

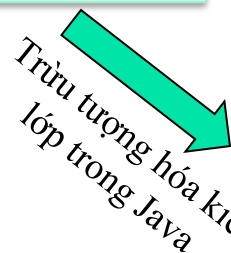
**Thông tin cần quản lý của 1 SV gồm:**

- mã số: Chuỗi ký tự
- họ và tên: Chuỗi ký tự
- năm sinh: Số nguyên
- điểm TB: Số thực



Trừu tượng hóa kiểu cấu trúc trong C

```
struct K_SinhVien{  
    char maSo[15];  
    char hoTen[30];  
    int namSinh;  
    double diemTB;  
}
```



Trừu tượng hóa kiểu lớp trong Java

```
class SinhVien{  
    private String maSo;  
    private String hoTen;  
    private int namSinh;  
    private double diemTB;  
  
    public void nhapTT(){...}  
    public void hienThiTT(){...}  
    ...  
}
```



# Bao bọc và che giấu thông tin

18

- ❖ Việc đóng gói dữ liệu và các hàm vào một đơn vị cấu trúc (gọi là lớp) được xem như một nguyên tắc **bao bọc thông tin**
- ❖ Kỹ thuật này cho phép xác định các vùng đặc trưng (riêng, công khai hay được bảo vệ bao gồm cả dữ liệu và các câu lệnh) nhằm điều khiển hoặc hạn chế những truy nhập tùy tiện của những đối tượng khác.
- ❖ Nguyên tắc bao bọc dữ liệu để ngăn cấm sự truy nhập trực tiếp trong lập trình được gọi là **sự che giấu thông tin**.



# Bao bọc và che giấu thông tin

19

| Employee                                                                          |                 |
|-----------------------------------------------------------------------------------|-----------------|
|  | name            |
|  | address         |
|  | salary          |
|  | thueMuon()      |
|  | duoiViec()      |
|  | deBatNhanVien() |

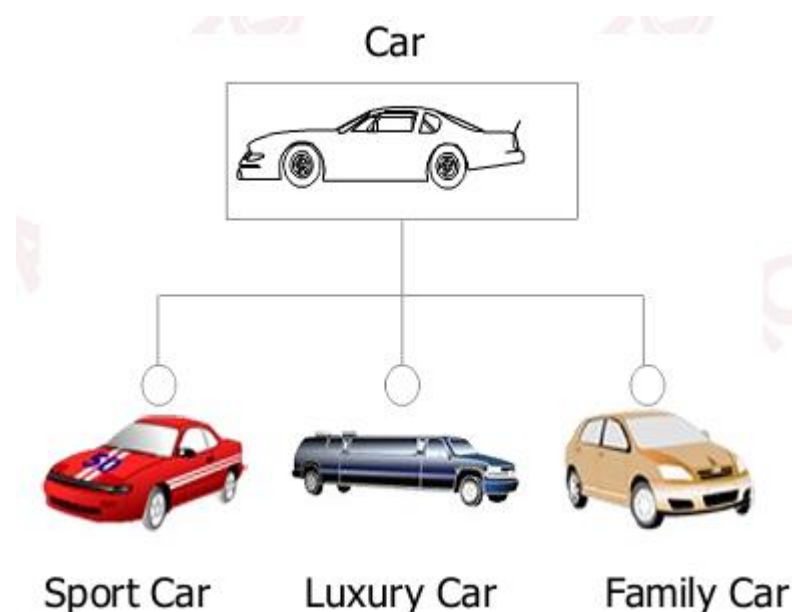
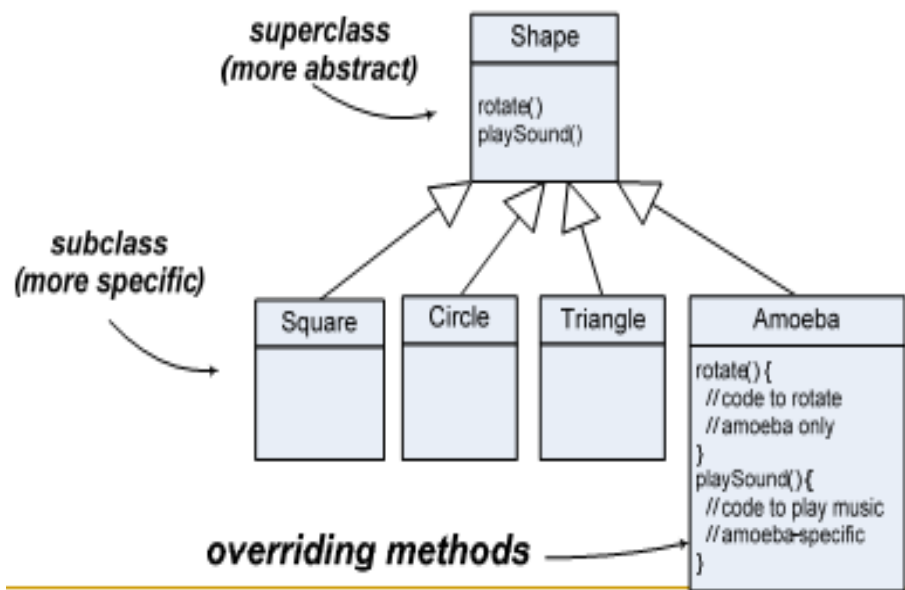
```
class Employee{  
    private String name;  
    private String address;  
    private double salary;  
  
    public void thueMuon(){  
        System.out.println("thueMuon method!");  
    }  
    public void duoiViec(){  
        System.out.println("duoiViec method!");  
    }  
    public void deBatNhanVien(){  
        System.out.println("deBatNhanVien method!");  
    }  
}
```



# Kế thừa và mở rộng

20

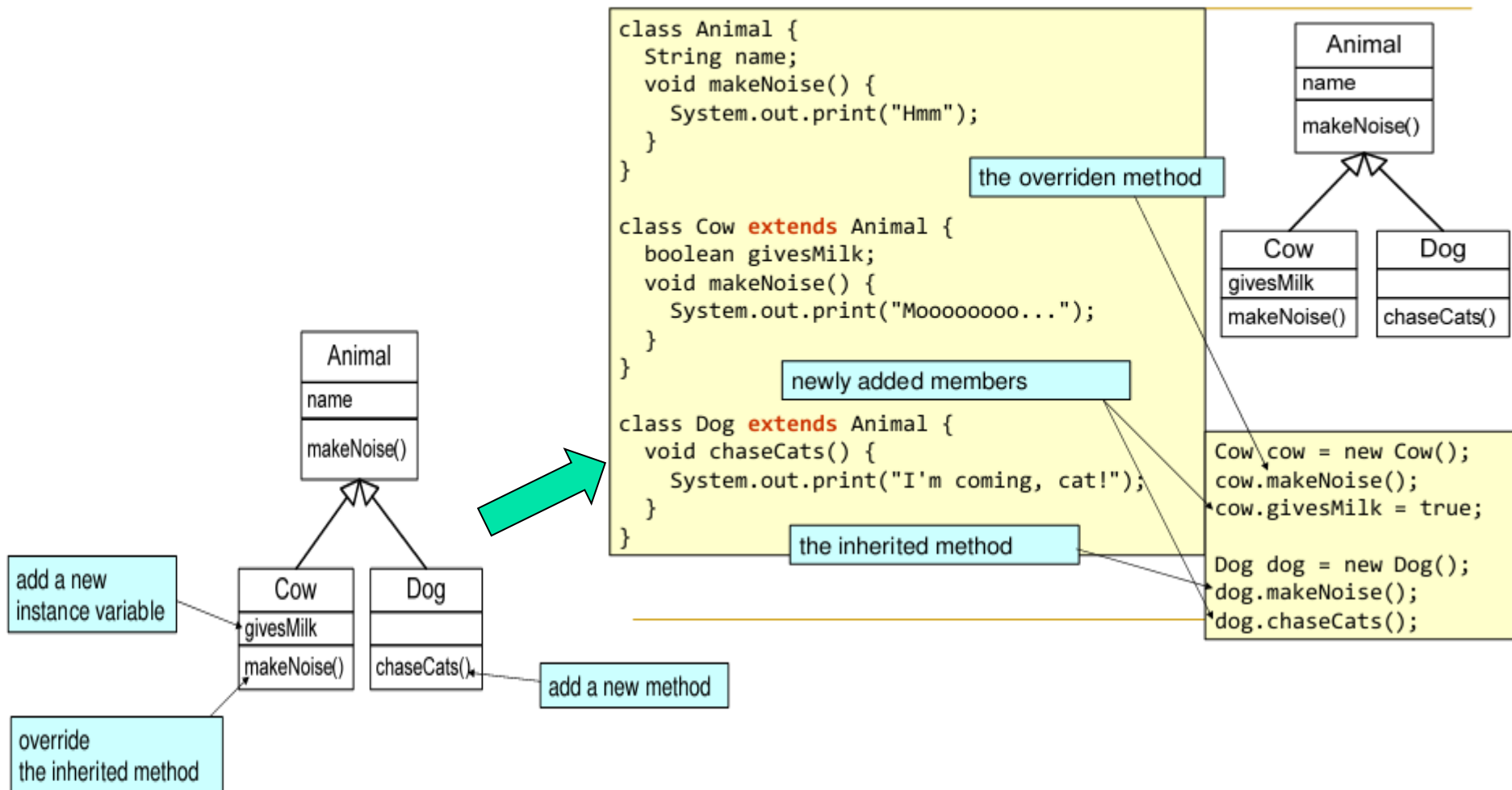
❖ **Kế thừa (Inheritance):** Cơ chế cho phép các đối tượng có thể chia sẻ hay mở rộng các thuộc tính hoặc phương thức của lớp khác mà không phải tiến hành định nghĩa lại.





# Kế thừa và mở rộng

21





# Kế thừa và mở rộng

22

- ❖ Lập trình hướng đối tượng đưa thêm những định nghĩa: lớp con (subclass), lớp cha (superclass), kế thừa (inherit), override.
- ❖ Lớp con kế thừa toàn bộ những biến của lớp cha. Ngoài ra, lớp con còn kế thừa một số phương thức của lớp cha.
- ❖ Lợi ích của kế thừa: lớp con cung cấp những phương thức chuyên biệt (code reuse). Lập trình viên có thể tạo ra những lớp abstract để định nghĩa những phương thức chung.



❖ **Đa hình:** Nhiều hình thức, nhiều kiểu tồn tại.

❖ **Tính đa hình thể hiện qua việc:** cùng một phương thức nhưng có nội dung thực hiện khác nhau trên các đối tượng khác nhau.

- Đa hình trong lập trình:
- Đa hình hàm (Nạp chồng hàm)
- Đa hình đối tượng:
  - Nhìn nhận đối tượng theo nhiều kiểu khác nhau
  - Các đối tượng khác nhau giải nghĩa thông điệp theo cách thức khác nhau

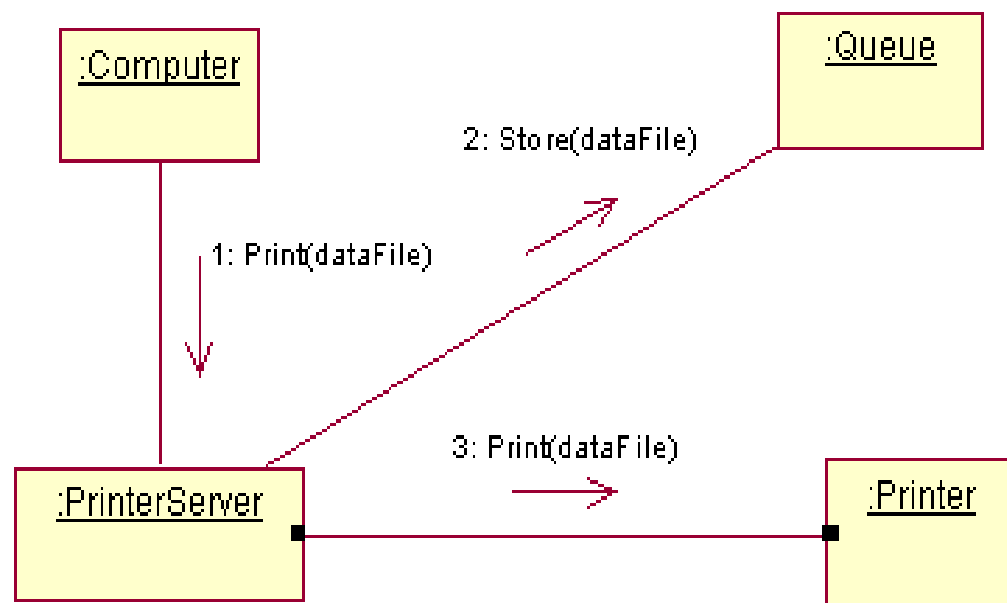




- ❖ Liên kết tĩnh: lời gọi hàm (phương thức) được quyết định khi biên dịch, do đó chỉ có một phiên bản của chương trình con được thực hiện. Ưu điểm về tốc độ
- ❖ Liên kết động: lời gọi hàm được quyết định khi thực hiện, phiên bản của phương thức phù hợp với đối tượng được gọi. Java mặc định sử dụng liên kết động.



- ❖ Truyền thông điệp cho một đối tượng chính là để yêu cầu thực hiện một công việc cụ thể nào đó, nghĩa là sử dụng những hàm tương ứng để xử lý dữ liệu đã được khai báo trong lớp đối tượng đó.





# Một số ngôn ngữ lập trình hỗ trợ lập trình HĐT

26

|               | Đặc trưng chính                                                               | Quản lý bộ nhớ               | Ứng dụng phổ biến                                                          | Cộng đồng hỗ trợ                           | Tính mở rộng                             |
|---------------|-------------------------------------------------------------------------------|------------------------------|----------------------------------------------------------------------------|--------------------------------------------|------------------------------------------|
| <b>Java</b>   | Độc lập nền tảng, bảo mật cao, hỗ trợ mạnh lập trình đa luồng                 | Tự động (Garbage Collection) | Phát triển web (Spring), di động (Android), ứng dụng doanh nghiệp          | Rất lớn, nhiều tài liệu và diễn đàn hỗ trợ | Dễ mở rộng với các framework             |
| <b>C++</b>    | Hiệu suất cao, quản lý bộ nhớ thủ công, hỗ trợ lập trình thủ tục và HĐT       | Thủ công                     | Ứng dụng hệ thống, game, phần mềm nhúng                                    | Lớn, nhưng tài liệu đôi khi phức tạp       | Rất linh hoạt, nhưng cần kiến thức sâu   |
| <b>Python</b> | Dễ học, cú pháp đơn giản, hỗ trợ lập trình HĐT và hàm                         | Tự động (Garbage Collection) | Khoa học dữ liệu, trí tuệ nhân tạo, phát triển web (Django, Flask)         | Rất lớn, nhiều thư viện mã nguồn mở        | Dễ mở rộng với các thư viện và framework |
| <b>C#</b>     | Tích hợp với Windows, hỗ trợ hiện đại, lập trình bất đồng bộ, LINQ            | Tự động (Garbage Collection) | Phát triển ứng dụng Windows, game (Unity), ứng dụng doanh nghiệp (ASP.NET) | Lớn, đặc biệt trong cộng đồng Microsoft    | Mở rộng tốt với .NET ecosystem           |
| <b>Ruby</b>   | Ngôn ngữ động, cú pháp đẹp, hỗ trợ mạnh lập trình HĐT, mạnh về lập trình meta | Tự động (Garbage Collection) | Phát triển web (Ruby on Rails)                                             | Trung bình, tài liệu và diễn đàn phong phú | Tốt trong môi trường web                 |
| <b>Swift</b>  | Hiệu suất cao, an toàn, dễ học, thiết kế hiện đại, quản lý bộ nhớ tự động     | Tự động (Garbage Collection) | Phát triển ứng dụng iOS, macOS                                             | Lớn, đặc biệt trong cộng đồng Apple        | Mở rộng tốt trong hệ sinh thái Apple     |



# Cài đặt, thiết lập môi trường phát triển ứng dụng Java

27

❖ Cài đặt JDK (Java Development Kit):  
<https://www.oracle.com/java/technologies/javase-downloads.html>

❖ Eclipse: <https://www.eclipse.org/downloads/>

❖ NetBean:  
<https://netbeans.apache.org/front/main/download/>



# Câu hỏi

28